LAB 8

Software Metrics & Complexity Analysis

Project: Digital Document Verification System

Subject	Software Engineering
Experiment	Software Metrics & Complexity Analysis
Date	28 October 2026

1. Aim

To study software metrics and analyze the complexity of selected modules in the Digital Document Verification System.

2. Objectives

- Understand software metrics and why they are used.
- Estimate basic size and quality metrics for the system.
- Calculate cyclomatic complexity using a control-flow graph formula.
- Identify modules that may need testing or refactoring.

3. Theory

Software metrics: Software metrics are measurable values used to estimate software size, complexity, quality, and maintainability. They help developers compare modules, plan testing, and identify code that may be difficult to maintain.

Lines of Code (LOC): LOC measures the number of source-code lines. It is easy to count, but it does not directly measure correctness or quality.

Cyclomatic Complexity (CC): Cyclomatic complexity estimates the number of linearly independent paths through a program. A common formula is CC = E − N + 2P, where E is the number of control-flow edges, N is the number of nodes, and P is the number of connected components (usually 1 for a single function).

Maintainability: Maintainability describes how easily software can be understood, corrected, and modified. Smaller functions, clear names, low duplication, and useful tests generally help.

Defect density: Defect density can be calculated as the number of known defects divided by software size, such as defects per 1,000 LOC. It is meaningful only when defect tracking and size measurement are consistent.

4. Complexity Estimation for System Modules

The following values are example estimates for a lab exercise. They should be replaced with values measured from the final source code.

Module	Estimated LOC	Decision points	CC (decisions + 1)	Comment
User login / authentication	80	3	4	Moderate branching; validate inputs and errors.
Document upload	95	4	5	Check file type, size, and upload failures.
Verification workflow	120	5	6	Most branching;

				prioritize path testing.
Admin review	90	3	4	Test approval, rejection, and missing data.
Notification service	55	2	3	Test success and delivery failure.

5. Sample Code and Calculation

Example: a simplified document-verification function.

```
function verifyDocument(document, user) {
  if (!user) return "unauthorized";
  if (!document) return "missing document";
  if (!isAllowedType(document.type)) return "invalid type";
  if (document.status === "verified") return "already verified";
  return compareDocumentDetails(document) ? "verified" : "needs review";
}
```

For a single connected function, a quick estimate is CC = number of decision points + 1. The four conditional checks give CC = 4 + 1 = 5.

Metric	Value	Explanation
Decision points	4	Four if/conditional decisions in the example.
Cyclomatic complexity	5	CC = number of decision points + 1 = 4 + 1.
Interpretation	Moderate	Five independent paths should be considered in test design.

6. Testing Recommendations

- Keep verification rules in small, focused functions.
- Validate uploaded files on both client and server sides.
- Add tests for missing users, missing documents, unsupported file types, already-verified documents, successful verification, and review-required cases.
- Log verification actions with the user, timestamp, and result.
- Measure actual LOC and complexity from the final source code; the values in this report are illustrative estimates, not measurements from a running repository.

7. Conclusion

Software metrics provide a practical way to discuss the size and complexity of a system. In this example, the verification workflow has the highest estimated complexity, so it should receive extra attention during testing and review. Actual metrics should be recalculated from the completed implementation.